Wycliffe Kipleting Rotich

Platform Engineer | Distributed Systems | Python, AWS, Kubernetes

Nairobi, Kenya | celestinerotich969@gmail.com | linkedin.com/in/rotichkipleting
github.com/wycliffRotich-dev/aethergrid | Live demo: aethergrid-dashboard.onrender.com

Open to remote roles
Available to start immediately

7+ yrs	802	72	29 s to 0.124 s
backend and distributed systems in production	passing tests in AetherGrid	architecture decision records (ADRs)	worst-case latency during a database outage

PROFESSIONAL SUMMARY

Platform engineer with 7+ years building backend and distributed systems on cloud platforms, focused on concurrency, race-condition debugging, execution ownership and failure recovery. Creator and maintainer of **AetherGrid**, an open-source distributed AI workload orchestrator covering scheduling, lease-based execution ownership, reconciliation, scoped API keys, hardened containers and tenant-scoped data access. Previously at Toyota and the Kenya Revenue Authority, working on systems that handle vehicle telemetry and national-scale financial data.

TECHNICAL SKILLS

Languages	Python, TypeScript, JavaScript, SQL
Backend	FastAPI, REST APIs, Async Python, Pydantic, PostgreSQL, psycopg, SQLite, Node.js
Distributed systems	Scheduling, leases, reconciliation, concurrency, race-condition debugging, retries with backoff and jitter, circuit breakers, idempotency, worker heartbeats, node draining, Kafka
Cloud and DevOps	AWS, ECS, Kubernetes, Terraform, Docker, Docker Compose, Docker Secrets, GitHub Actions, CI/CD, Dependabot
Security	API key authentication, scoped authorization, tenant isolation, container hardening (non-root, read-only filesystem, dropped capabilities)
Frontend	React 19, TypeScript, Vite, TailwindCSS, client-side routing, Playwright
Testing	TDD, pytest, contract, integration, API and end-to-end testing, mutation verification
Design	Clean Architecture, DDD, SOLID, aggregates, value objects, state machines, repository pattern, domain events, ADRs
AI and ML	LLM evaluation, code quality assessment, AI training pipelines, data annotation and QA

PROFESSIONAL EXPERIENCE

Founder and Platform Engineer

Jan 2026 - Present

AetherGrid | Open-source AI workload orchestrator | Nairobi

- Designed and built AetherGrid end to end with Python, FastAPI, PostgreSQL (raw psycopg, no ORM), React, TypeScript, Docker Compose and GitHub Actions. Four layers with a domain core that has no infrastructure imports. **802 passing tests** across unit, contract, integration, API and end-to-end, and **72 ADRs**.
- Built a multi-dimensional best-fit scheduler across CPU, memory and VRAM, with time-bound leases renewed for the full run of a job. Reconciliation reclaims work from dead workers, expired leases, offline nodes and jobs abandoned mid-cancellation, within a bounded retry budget.
- Found and closed a recurring **race-condition class** by fencing lease release, outcome reporting and renewal on lease identity, then fencing reconciliation's own deletes by expiry. Each fix began with a test that reproduced the bug and was verified against real PostgreSQL.
- Built a real subprocess engine with enforced timeouts and two-stage termination (SIGTERM, then SIGKILL), with cancellation delivered over the lease-renewal channel and an explicit CANCELLING state. Built a standalone worker agent with capped exponential backoff and full jitter that re-registers after control plane restarts, one API key per agent.
- Kept the API responsive during database outages by moving reconciliation off the event loop, splitting liveness from readiness and bounding the pool timeout to 5 seconds. Worst-case latency on an unrelated endpoint dropped from **about 29 s to 0.124 s**, measured against a stopped Postgres container.
- Secured the API with key authentication on every route, scoped keys (jobs:execute, keys:manage), key ownership and locally minted first keys. Hardened containers: non-root, read-only root filesystem, all capabilities dropped, no-new-privileges, pinned digests, database password as a Docker secret. A CI workflow asserts each property, and each check was proven to fail on a deliberate break.
- Implemented **tenant scoping** for API keys, nodes and jobs. The tenant comes from the credential, never the request, and another tenant's resource answers like a missing one. Workers, leases and events are the next stage.

- Built a resilient HTTP client (retries, circuit breaker, OAuth2, mTLS, replay of only safe requests) and a React 19 operator dashboard with a live event feed, per-job history and a Playwright end-to-end test in CI.

LLM Technical Trainer / AI Training Specialist
Aug 2025 - Present

Mercor | Remote

- Evaluate LLM-generated code for correctness, architecture, concurrency and API behavior.
- Review backend implementations where the happy path works but the system breaks under real conditions, finding issues in state management, async execution and race conditions.
- Write technical feedback that improves model reasoning and code quality.

AI Data Specialist, Video Annotation and Quality Review
Dec 2024 - May 2025

Atlas Capture

- Annotated and reviewed video datasets for machine-learning workflows using CVAT, Labelbox, Scale, V7 and Encord.
- Performed QA reviews and resolved inconsistent or ambiguous annotations.

Software Engineer, Cloud Platforms
Jan 2024 - Nov 2024

Toyota | Nairobi

- Fixed concurrency problems in worker queues, resolving race conditions in Kafka-based ingestion pipelines for vehicle and telematics event data.
- Built high-volume Kafka data ingestion pipelines with asynchronous downstream processing.
- Deployed and orchestrated containerized workloads on Kubernetes and ECS across AWS environments.
- Provisioned cloud infrastructure with Terraform, improving resource allocation and reducing provisioning time.
- Designed REST APIs with strict DTO validation and refactored tightly coupled components to separate business logic from framework code.

Backend Cloud Engineer
Nov 2018 - Oct 2023

Kenya Revenue Authority (KRA) | Nairobi

- Worked with senior engineers on production revenue systems handling real financial data at national scale.
- Traced race conditions where concurrent requests interfered with shared state, isolating root causes across application logic, shared state and database interactions.
- Investigated timing-dependent bugs that were difficult to reproduce through standard request-response testing.

EDUCATION

Master of Science, Computer Science
Jan 2022 - Dec 2024

United States International University-Africa (USIU-Africa), part-time

Bachelor of Engineering, Computer Software Engineering
Aug 2014 - Nov 2018

The Technical University of Kenya